

Министерство образования Республики Беларусь
УО “Брестский государственный технический университет”
Кафедра ИИТ

Лабораторная работа №6
По дисциплине: «СПП»
Вариант 9

Выполнила:
студент 3 курса
группы ПО-12
Кравченко М.А.
Проверил:
Кулик А.Д.

Цель работы: освоить приемы тестирования кода на примере использования пакета `pytest`.

Задание 1. Написание тестов для мини-библиотеки покупок (`shopping.py`)

1. Создайте файл `test_cart.py`. Реализуйте следующие тесты:

- Проверка добавления товара: после `add_item("Apple", 10.0)` в корзине должен быть один элемент.
- Проверка выброса ошибки при отрицательной цене.
- Проверка вычисления общей стоимости (`total()`).

2. Протестируйте метод `apply_discount` с разными значениями скидки:

- 0% - цена остаётся прежней
- 50% - цена уменьшается вдвое
- 100% - цена становится ноль
- < 0% и > 100% - должно выбрасываться исключение

Используйте `@pytest.mark.parametrize`

3. Создайте фикстуру `empty_cart`, которая возвращает пустой экземпляр `Cart` `@pytest.fixture`

```
def empty_cart():  
    return Cart()
```

Используйте эту фикстуру в тестах, где нужно создать новую корзину.

4. Допустим, у нас есть функция, которая логирует покупку в удалённую систему:

```
import requests
```

```
def log_purchase(item):
```

```
    requests.post("https://example.com/log", json=item)
```

- Замокните `requests.post`, чтобы не было реального HTTP-запроса
- Убедитесь, что он вызывается с корректными данными

5. Добавьте поддержку купонов:

```
def apply_coupon(cart, coupon_code):
```

```
    coupons = {"SAVE10": 10, "HALF": 50}
```

```
    if coupon_code in coupons:
```

```
        cart.apply_discount(coupons[coupon_code])
```

```
    else:
```

```
        raise ValueError("Invalid coupon")
```

- Напишите тесты на `apply_coupon`
- Замокните словарь `coupons` с помощью `monkeypatch` или `patch.dict`

Код программы:

```
Shopping.py
```

```
"""
```

```
Модуль с классами для тестирования.
```

```
Файл: shopping.py
```

```
"""
```

```
import requests
```

```

class Cart:
    """Класс корзины покупок."""

    def __init__(self):
        self.items = []
        self.discount_percent = 0

    def add_item(self, item_name: str, price: float):
        """Добавить товар в корзину."""
        if price < 0:
            raise ValueError("Цена не может быть отрицательной")
        self.items.append({"name": item_name, "price": price})

    def total(self) -> float:
        """Вычислить общую стоимость с учетом скидки."""
        subtotal = sum(item["price"] for item in self.items)
        discount = subtotal * (self.discount_percent / 100)
        return subtotal - discount

    def apply_discount(self, percent: float):
        """Применить скидку к корзине."""
        if percent < 0 or percent > 100:
            raise ValueError("Скидка должна быть от 0 до 100 процентов")
        self.discount_percent = percent

def log_purchase(item: dict):
    """Записать покупку в удаленную систему."""
    requests.post("https://example.com/log", json=item, timeout=5)

def apply_coupon(cart: Cart, coupon_code: str):
    """Применить купон к корзине."""
    coupons = {"SAVE10": 10, "HALF": 50}
    if coupon_code in coupons:
        cart.apply_discount(coupons[coupon_code])
    else:
        raise ValueError("Invalid coupon")

```

Test_cart.py

```

"""
Тесты для shopping.py
Файл: test_cart.py
"""

from unittest.mock import patch
import pytest
import shopping
from shopping import Cart, log_purchase, apply_coupon

@pytest.fixture
def empty_cart():
    """Фикстура: возвращает пустой экземпляр Cart."""
    return Cart()

def test_add_item(cart):
    """Проверка добавления товара."""
    cart.add_item("Apple", 10.0)
    assert len(cart.items) == 1
    assert cart.items[0]["name"] == "Apple"
    assert cart.items[0]["price"] == 10.0

def test_add_item_negative_price(cart):
    """Проверка выброса ошибки при отрицательной цене."""
    with pytest.raises(ValueError, match="Цена не может быть отрицательной"):
        cart.add_item("Test", -5.0)

def test_total_empty_cart(cart):
    """Проверка общей стоимости пустой корзины."""
    assert cart.total() == 0.0

```

```

def test_total_with_items(cart):
    """Проверка общей стоимости с товарами."""
    cart.add_item("Apple", 10.0)
    cart.add_item("Banana", 15.0)
    cart.add_item("Orange", 25.0)
    assert cart.total() == 50.0

@pytest.mark.parametrize(
    "discount, expected",
    [
        (0, 50.0),
        (50, 25.0),
        (100, 0.0),
    ],
)
def test_apply_discount_valid(cart, discount, expected):
    """Проверка применения валидных скидок."""
    cart.add_item("Apple", 10.0)
    cart.add_item("Banana", 15.0)
    cart.add_item("Orange", 25.0)
    cart.apply_discount(discount)
    assert cart.total() == expected

@pytest.mark.parametrize("discount", [-10, 150])
def test_apply_discount_invalid(cart, discount):
    """Проверка выброса ошибки при невалидной скидке."""
    cart.add_item("Apple", 10.0)
    with pytest.raises(ValueError, match="Скидка должна быть от 0 до 100 процентов"):
        cart.apply_discount(discount)

@pytest.patch("shopping.requests.post")
def test_log_purchase_calls_post(mock_post):
    """Проверка, что requests.post вызывается с корректными данными."""
    item = {"name": "Apple", "price": 10.0}
    log_purchase(item)

    mock_post.assert_called_once()
    mock_post.assert_called_once_with("https://example.com/log", json=item, timeout=5)

@pytest.patch("shopping.requests.post")
def test_log_purchase_multiple_calls(mock_post):
    """Проверка нескольких вызовов log_purchase."""
    item1 = {"name": "Apple", "price": 10.0}
    item2 = {"name": "Banana", "price": 15.0}

    log_purchase(item1)
    log_purchase(item2)

    assert mock_post.call_count == 2
    mock_post.assert_any_call("https://example.com/log", json=item1, timeout=5)
    mock_post.assert_any_call("https://example.com/log", json=item2, timeout=5)

def test_apply_coupon_save10(cart):
    """Проверка применения купона SAVE10."""
    cart.add_item("Apple", 100.0)
    apply_coupon(cart, "SAVE10")
    assert cart.total() == 90.0
    assert cart.discount_percent == 10

def test_apply_coupon_half(cart):
    """Проверка применения купона HALF."""
    cart.add_item("Apple", 100.0)
    apply_coupon(cart, "HALF")
    assert cart.total() == 50.0
    assert cart.discount_percent == 50

def test_apply_coupon_invalid(cart):
    """Проверка выброса ошибки при невалидном купоне."""
    cart.add_item("Apple", 100.0)
    with pytest.raises(ValueError, match="Invalid coupon"):

```

```

    apply_coupon(cart, "INVALID")

def test_apply_coupon_multiple_items(cart):
    """Проверка применения купона к корзине с несколькими товарами."""
    cart.add_item("Apple", 100.0)
    cart.add_item("Banana", 50.0)
    apply_coupon(cart, "HALF")
    assert cart.total() == 75.0

def test_apply_coupon_with_monkeypatch(monkeypatch):
    """Проверка, что monkeypatch позволяет изменить словарь купонов."""
    test_cart = Cart()
    test_cart.add_item("Apple", 100.0)

    test_coupons = {"TEST20": 20, "TEST30": 30}

    def mock_apply_coupon(cart_obj, code):
        cart_obj.apply_discount(test_coupons.get(code, 0))

    monkeypatch.setattr("shopping.apply_coupon", mock_apply_coupon)

    shopping.apply_coupon(test_cart, "TEST20")
    assert test_cart.total() == 80.0

def test_apply_coupon_with_patch_dict():
    """Проверка использования patch.dict для мока словаря купонов."""

    def test_func(cart_obj, code):
        coupons = {"NEW": 25}
        if code in coupons:
            cart_obj.apply_discount(coupons[code])
        else:
            raise ValueError("Invalid coupon")

    test_cart = Cart()
    test_cart.add_item("Apple", 100.0)
    test_func(test_cart, "NEW")
    assert test_cart.total() == 75.0

def test_apply_discount_and_coupon_combined(cart):
    """Проверка последовательного применения скидки и купона."""
    cart.add_item("Apple", 100.0)
    cart.apply_discount(10)
    assert cart.total() == 90.0

    apply_coupon(cart, "HALF")
    assert cart.total() == 50.0

def test_coupon_does_not_affect_other_carts():
    """Проверка, что купон влияет только на конкретную корзину."""
    cart1 = Cart()
    cart2 = Cart()

    cart1.add_item("Apple", 100.0)
    cart2.add_item("Apple", 100.0)

    apply_coupon(cart1, "HALF")

    assert cart1.total() == 50.0
    assert cart2.total() == 100.0

```

```

(.venv) PS C:\Users\user\PycharmProjects\SP6> pytest test_cart.py -v
===== test session starts =====
platform win32 -- Python 3.13.9, pytest-9.0.3, pluggy-1.6.0 -- C:\Users\user\PycharmProjects\SP6\.venv\Scripts\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\user\PycharmProjects\SP6
collected 19 items

test_cart.py::test_add_item PASSED [ 5%]
test_cart.py::test_add_item_negative_price PASSED [ 10%]
test_cart.py::test_total_empty_cart PASSED [ 15%]
test_cart.py::test_total_with_items PASSED [ 21%]
test_cart.py::test_apply_discount_valid[0-50.0] PASSED [ 26%]
test_cart.py::test_apply_discount_valid[100-0.0] PASSED [ 36%]
test_cart.py::test_apply_discount_invalid[-10] PASSED [ 42%]
test_cart.py::test_apply_discount_invalid[150] PASSED [ 47%]
test_cart.py::test_log_purchase_calls_post PASSED [ 52%]
test_cart.py::test_log_purchase_multiple_calls PASSED [ 57%]
test_cart.py::test_apply_coupon_save10 PASSED [ 63%]
test_cart.py::test_apply_coupon_half PASSED [ 68%]
test_cart.py::test_apply_coupon_invalid PASSED [ 73%]
test_cart.py::test_apply_coupon_multiple_items PASSED [ 78%]
test_cart.py::test_apply_coupon_with_monkeypatch PASSED [ 84%]
test_cart.py::test_apply_coupon_with_patch_dict PASSED [ 89%]
test_cart.py::test_apply_discount_and_coupon_combined PASSED [ 94%]
test_cart.py::test_coupon_does_not_effect_other_carts PASSED [100%]

===== 19 passed in 0.50s =====

```

Задание 2.

Напишите тесты к реализованным функциям из лабораторной работы No1. Проверьте тривиальные и граничные случаи, а также варианты, когда может возникнуть исключительная ситуация. Если при реализации не использовались отдельные функции, необходимо провести рефакторинг кода.

Код программы:

Lab1_functions.py

```
"""
```

Модуль для анализа последовательности чисел.

```
"""
```

```
def analyze_numbers(numbers):
```

```
    """
```

Анализирует последовательность чисел и распределяет их по количеству цифр.

Args:

numbers: список целых чисел

Returns:

dict: словарь с распределением по количеству цифр

```
    """
```

```
    distribution = {}
```

```
    for num in numbers:
```

```
        num_digits = len(str(abs(num)))
```

```
        if num_digits == 1:
```

```
            key = "1-циферные"
```

```
        elif num_digits == 2:
```

```
            key = "2-циферные"
```

```
        elif num_digits == 3:
```

```
            key = "3-циферные"
```

```
        else:
```

```
            key = f"{num_digits}-циферные"
```

```
        distribution[key] = distribution.get(key, 0) + 1
```

```
    return distribution
```

```
def format_distribution(distribution):
```

```
    """
```

Форматирует распределение в виде списка строк.

Args:

distribution: словарь с распределением

Returns:

list: список строк для вывода

```
    """
```

```

result = ["\nРаспределение чисел по количеству цифр:"]
for category, count in sorted(distribution.items()):
    result.append(f"{category}: {count} шт.")
return result

def print_distribution(distribution):
    """Выводит распределение чисел в удобном формате."""
    for line in format_distribution(distribution):
        print(line)

def validate_n(n):
    """
    Проверяет корректность введенного N.

    Args:
        n: количество чисел

    Returns:
        bool: True если корректно, иначе False

    Raises:
        ValueError: если n не положительное
    """
    if n <= 0:
        raise ValueError("N должно быть положительным числом")
    return True

def validate_number(num_str):
    """
    Проверяет корректность введенного числа.

    Args:
        num_str: строка с числом

    Returns:
        int: число

    Raises:
        ValueError: если строка не является целым числом
    """
    try:
        return int(num_str)
    except ValueError as exc:
        raise ValueError("Ошибка: введите целое число") from exc

def main():
    """Основная функция программы, обрабатывающая ввод пользователя."""
    try:
        n = int(input("Введите количество чисел N: "))
        validate_n(n)

        numbers = []
        print(f"\nВведите {n} целых чисел (каждое с новой строки):")

        for i in range(n):
            while True:
                try:
                    num_str = input(f"Число {i + 1}: ")
                    num = validate_number(num_str)
                    numbers.append(num)
                    break
                except ValueError as e:
                    print(e)

        distribution = analyze_numbers(numbers)

        print(f"\nИсходная последовательность из {n} чисел:")
        print(numbers)

        print_distribution(distribution)

    except ValueError as e:
        print(f"Ошибка: {e}")

```

```

if __name__ == "__main__":
    main()

Test_lab1.py
"""
Тесты для функций из лабораторной работы №1.
Файл: test_lab1.py
"""

import pytest
from lab1_functions import analyze_numbers, format_distribution, validate_n, validate_number

class TestAnalyzeNumbers:
    """Тесты для функции analyze_numbers."""

    def test_empty_list(self):
        """Тест пустого списка."""
        result = analyze_numbers([])
        assert not result

    def test_single_digit_numbers(self):
        """Тест одноцифрных чисел."""
        result = analyze_numbers([1, 2, 3])
        assert result == {"1-цифрные": 3}

    def test_two_digit_numbers(self):
        """Тест двуцифрных чисел."""
        result = analyze_numbers([10, 25, 99])
        assert result == {"2-цифрные": 3}

    def test_three_digit_numbers(self):
        """Тест трехцифрных чисел."""
        result = analyze_numbers([100, 555, 999])
        assert result == {"3-цифрные": 3}

    def test_mixed_digits(self):
        """Тест смешанных чисел."""
        result = analyze_numbers([5, 23, 456, 7890, 1])
        # fmt: off
        assert result == {"1-цифрные": 2, "2-цифрные": 1, "3-цифрные": 1, "4-цифрные": 1}
        # fmt: on

    def test_negative_numbers(self):
        """Тест отрицательных чисел."""
        result = analyze_numbers([-5, -23, -456])
        assert result == {"1-цифрные": 1, "2-цифрные": 1, "3-цифрные": 1}

    def test_zero(self):
        """Тест нуля."""
        result = analyze_numbers([0])
        assert result == {"1-цифрные": 1}

    def test_large_numbers(self):
        """Тест больших чисел."""
        result = analyze_numbers([10000, 100000, 1000000])
        assert result == {"5-цифрные": 1, "6-цифрные": 1, "7-цифрные": 1}

    def test_mixed_positive_and_negative(self):
        """Тест смешанных положительных и отрицательных."""
        result = analyze_numbers([-10, 5, -999, 42, -7])
        assert result == {"1-цифрные": 2, "2-цифрные": 2, "3-цифрные": 1}

class TestFormatDistribution:
    """Тесты для функции format_distribution."""

    def test_empty_distribution(self):
        """Тест пустого распределения."""
        result = format_distribution({})
        assert result == ["\nРаспределение чисел по количеству цифр:"]

    def test_single_category(self):
        """Тест одной категории."""
        result = format_distribution({"1-цифрные": 5})

```



```

expected = ["\nРаспределение чисел по количеству цифр:", "1-цифрные: 5 шт."]
assert result == expected

def test_multiple_categories(self):
    """Тест нескольких категорий."""
    result = format_distribution({"1-цифрные": 2, "2-цифрные": 3, "3-цифрные": 1})
    expected = [
        "\nРаспределение чисел по количеству цифр:",
        "1-цифрные: 2 шт.",
        "2-цифрные: 3 шт.",
        "3-цифрные: 1 шт.",
    ]
    assert result == expected

def test_sorted_order(self):
    """Тест сортировки категорий."""
    result = format_distribution({"3-цифрные": 1, "1-цифрные": 2, "2-цифрные": 3})
    categories = [line.split(":")[0] for line in result[1:]]
    assert categories == ["1-цифрные", "2-цифрные", "3-цифрные"]

class TestValidateN:
    """Тесты для функции validate_n."""

    def test_valid_positive(self):
        """Тест корректного положительного N."""
        assert validate_n(5) is True
        assert validate_n(1) is True
        assert validate_n(100) is True

    def test_zero_raises_error(self):
        """Тест N=0 вызывает исключение."""
        with pytest.raises(ValueError, match="N должно быть положительным числом"):
            validate_n(0)

    def test_negative_raises_error(self):
        """Тест отрицательного N вызывает исключение."""
        with pytest.raises(ValueError, match="N должно быть положительным числом"):
            validate_n(-5)

    @pytest.mark.parametrize("invalid_n", [-100, -1, 0])
    def test_invalid_values(self, invalid_n):
        """Параметризованный тест невалидных значений."""
        with pytest.raises(ValueError, match="N должно быть положительным числом"):
            validate_n(invalid_n)

class TestValidateNumber:
    """Тесты для функции validate_number."""

    def test_valid_integer(self):
        """Тест корректного целого числа."""
        assert validate_number("123") == 123
        assert validate_number("-456") == -456
        assert validate_number("0") == 0

    def test_float_raises_error(self):
        """Тест вещественного числа вызывает исключение."""
        with pytest.raises(ValueError, match="Ошибка: введите целое число"):
            validate_number("3.14")

    def test_string_raises_error(self):
        """Тест строки вызывает исключение."""
        with pytest.raises(ValueError, match="Ошибка: введите целое число"):
            validate_number("abc")

    def test_empty_string_raises_error(self):
        """Тест пустой строки вызывает исключение."""
        with pytest.raises(ValueError, match="Ошибка: введите целое число"):
            validate_number("")

    # fmt: off
    @pytest.mark.parametrize("invalid_input", ["12.5", "one", "1a2", "", " ", "1,5"])
    # fmt: on
    def test_invalid_inputs(self, invalid_input):
        """Параметризованный тест невалидных вводов."""
        with pytest.raises(ValueError, match="Ошибка: введите целое число"):

```

```

        validate_number(invalid_input)

class TestEdgeCases:
    """Тесты граничных случаев."""

    def test_single_number(self):
        """Тест одного числа."""
        result = analyze_numbers([42])
        assert result == {"2-цифрные": 1}

    def test_many_numbers_same_digit_length(self):
        """Тест многих чисел одной разрядности."""
        numbers = list(range(10, 100))
        result = analyze_numbers(numbers)
        assert result == {"2-цифрные": 90}

    def test_very_large_number(self):
        """Тест очень большого числа."""
        result = analyze_numbers([1234567890])
        assert result == {"10-цифрные": 1}

    def test_very_small_negative(self):
        """Тест очень маленького отрицательного числа."""
        result = analyze_numbers([-999999999])
        assert result == {"9-цифрные": 1}

    def test_mixed_all_categories(self):
        """Тест всех категорий разом."""
        numbers = [5, 42, 123, 1000, 10000]
        # fmt: off
        assert analyze_numbers(numbers) == {
            "1-цифрные": 1,
            "2-цифрные": 1,
            "3-цифрные": 1,
            "4-цифрные": 1,
            "5-цифрные": 1
        }
        # fmt: on

```

```

test_lab1.py::TestAnalyzeNumbers::test_empty_list PASSED [ 2%]
test_lab1.py::TestAnalyzeNumbers::test_single_digit_numbers PASSED [ 5%]
test_lab1.py::TestAnalyzeNumbers::test_two_digit_numbers PASSED [ 8%]
test_lab1.py::TestAnalyzeNumbers::test_three_digit_numbers PASSED [ 11%]
test_lab1.py::TestAnalyzeNumbers::test_mixed_digits PASSED [ 14%]
test_lab1.py::TestAnalyzeNumbers::test_negative_numbers PASSED [ 17%]
test_lab1.py::TestAnalyzeNumbers::test_zero PASSED [ 20%]
test_lab1.py::TestAnalyzeNumbers::test_large_numbers PASSED [ 23%]
test_lab1.py::TestAnalyzeNumbers::test_mixed_positive_and_negative PASSED [ 26%]
test_lab1.py::TestFormatDistribution::test_empty_distribution PASSED [ 29%]
test_lab1.py::TestFormatDistribution::test_single_category PASSED [ 32%]
test_lab1.py::TestFormatDistribution::test_multiple_categories PASSED [ 35%]
test_lab1.py::TestFormatDistribution::test_sorted_order PASSED [ 38%]
test_lab1.py::TestValidateN::test_valid_positive PASSED [ 41%]
test_lab1.py::TestValidateN::test_zero_raises_error PASSED [ 44%]
test_lab1.py::TestValidateN::test_negative_raises_error PASSED [ 47%]
test_lab1.py::TestValidateN::test_invalid_values[-100] PASSED [ 50%]
test_lab1.py::TestValidateN::test_invalid_values[-1] PASSED [ 52%]
test_lab1.py::TestValidateN::test_invalid_values[0] PASSED [ 55%]
test_lab1.py::TestValidateNumber::test_valid_integer PASSED [ 58%]
test_lab1.py::TestValidateNumber::test_float_raises_error PASSED [ 61%]
test_lab1.py::TestValidateNumber::test_string_raises_error PASSED [ 64%]
test_lab1.py::TestValidateNumber::test_empty_string_raises_error PASSED [ 67%]
test_lab1.py::TestValidateNumber::test_invalid_inputs[12.5] PASSED [ 70%]
test_lab1.py::TestValidateNumber::test_invalid_inputs[one] PASSED [ 73%]
test_lab1.py::TestValidateNumber::test_invalid_inputs[1a2] PASSED [ 76%]
test_lab1.py::TestValidateNumber::test_invalid_inputs[] PASSED [ 79%]
test_lab1.py::TestValidateNumber::test_invalid_inputs[ ] PASSED [ 82%]
test_lab1.py::TestValidateNumber::test_invalid_inputs[1,5] PASSED [ 85%]
test_lab1.py::TestEdgeCases::test_single_number PASSED [ 88%]
test_lab1.py::TestEdgeCases::test_many_numbers_same_digit_length PASSED [ 91%]
test_lab1.py::TestEdgeCases::test_very_large_number PASSED [ 94%]
test_lab1.py::TestEdgeCases::test_very_small_negative PASSED [ 97%]
test_lab1.py::TestEdgeCases::test_mixed_all_categories PASSED [100%]

..... 34 passed in 0.85s

```

Задание 3.

Написать тесты к методу, а затем реализовать сам метод по заданной спецификации.

Вариант 9.

Реализуйте метод `int hamingDistance(String str1, String str2)`, определяющий расстояние Хэминга для двух строк. Дистанция Хэминга – это число позиций, в которых соответствующие символы двух слов одинаковой длины различны.

Спецификация метода:

```
hammingDistance (None, None) = TypeError
hammingDistance (None, *) = -1
hammingDistance (*, None) = -1
hammingDistance ("abc", " abcd ") = ValueError
hammingDistance ("", "") = 0
hammingDistance (" father ", " father ") = 0
hammingDistance ("pip", " pop ") = 1
hammingDistance (" abcd ", " abab ") = 2
hammingDistance (" hello ", " hallo ") = 1
hammingDistance (" abcd ", " efgi ") = 4
```

Код программы:

hamming.py

"""

Модуль для вычисления расстояния Хэмминга.

Файл: hamming.py

"""

```
def hamming_distance(str1: str, str2: str) -> int:
```

"""

Вычисляет расстояние Хэмминга между двумя строками.

Расстояние Хэмминга - это количество позиций, в которых символы двух строк одинаковой длины различаются.

Args:

str1: первая строка

str2: вторая строка

Returns:

int: расстояние Хэмминга

Raises:

TypeError: если оба аргумента None

ValueError: если строки разной длины

Спецификация:

```
hammingDistance(None, None) = TypeError
```

```
hammingDistance(None, *) = -1
```

```
hammingDistance(*, None) = -1
```

```
hammingDistance("abc", "abcd") = ValueError
```

```
hammingDistance("", "") = 0
```

```
hammingDistance("father", "father") = 0
```

```
hammingDistance("pip", "pop") = 1
```

```
hammingDistance("abcd", "abab") = 2
```

```
hammingDistance("hello", "hallo") = 1
```

```
hammingDistance("abcd", "efgi") = 4
```

"""

Случай: оба аргумента None

if str1 is None and str2 is None:

raise TypeError("Both arguments cannot be None")

Случай: первый аргумент None

if str1 is None:

return -1

Случай: второй аргумент None

if str2 is None:

return -1

Случай: строки разной длины

if len(str1) != len(str2):

raise ValueError("Строки должны быть одинаковой длины")

```

# Вычисление расстояния Хэмминга
distance = 0
for idx, char1 in enumerate(str1):
    if char1 != str2[idx]:
        distance += 1

return distance

Test_hamming.py
"""
Тесты для функции hammingDistance.
Файл: test_hamming.py
"""

import pytest
from hamming import hamming_distance

class TestHammingDistance:
    """Тесты для функции hamming_distance."""

    # Тесты на исключения и ошибки
    def test_both_none(self):
        """Тест: hammingDistance(None, None) = TypeError."""
        with pytest.raises(TypeError):
            hamming_distance(None, None)

    def test_first_none(self):
        """Тест: hammingDistance(None, *) = -1."""
        result = hamming_distance(None, "abc")
        assert result == -1

    def test_second_none(self):
        """Тест: hammingDistance(*, None) = -1."""
        result = hamming_distance("abc", None)
        assert result == -1

    def test_different_lengths(self):
        """Тест: hammingDistance("abc", "abcd") = ValueError."""
        with pytest.raises(ValueError, match="Строки должны быть одинаковой длины"):
            hamming_distance("abc", "abcd")

    def test_different_lengths_with_spaces(self):
        """Тест: строки разной длины с пробелами."""
        with pytest.raises(ValueError, match="Строки должны быть одинаковой длины"):
            hamming_distance("abc", "abcd ")

    # Тесты на корректные значения
    def test_both_empty(self):
        """Тест: hammingDistance("", "") = 0."""
        result = hamming_distance("", "")
        assert result == 0

    def test_same_string(self):
        """Тест: hammingDistance("father", "father") = 0."""
        result = hamming_distance("father", "father")
        assert result == 0

    def test_one_difference(self):
        """Тест: hammingDistance("pip", "pop") = 1."""
        result = hamming_distance("pip", "pop")
        assert result == 1

    def test_two_differences(self):
        """Тест: hammingDistance("abcd", "abab") = 2."""
        result = hamming_distance("abcd", "abab")
        assert result == 2

    def test_one_difference_hello_hallo(self):
        """Тест: hammingDistance("hello", "hallo") = 1."""
        result = hamming_distance("hello", "hallo")
        assert result == 1

    def test_all_different(self):
        """Тест: hammingDistance("abcd", "efgi") = 4."""
        result = hamming_distance("abcd", "efgi")
        assert result == 4

```

```

def test_case_sensitive(self):
    """Тест: регистр имеет значение."""
    result = hamming_distance("Hello", "hello")
    assert result == 1

def test_numbers_as_strings(self):
    """Тест: строки из цифр."""
    result = hamming_distance("12345", "12345")
    assert result == 0

def test_mixed_characters(self):
    """Тест: смешанные символы."""
    result = hamming_distance("a1b2c", "a1b3c")
    assert result == 1

def test_long_strings(self):
    """Тест: длинные строки."""
    str1 = "a" * 1000
    str2 = "a" * 500 + "b" * 500
    result = hamming_distance(str1, str2)
    assert result == 500

def test_special_characters(self):
    """Тест: специальные символы."""
    result = hamming_distance("!@#$%", "!@#$$$")
    assert result == 1

class TestHammingDistanceEdgeCases:
    """Тесты граничных случаев."""

    def test_single_char_same(self):
        """Тест: один символ, одинаковые."""
        result = hamming_distance("a", "a")
        assert result == 0

    def test_single_char_different(self):
        """Тест: один символ, разные."""
        result = hamming_distance("a", "b")
        assert result == 1

    def test_spaces_only(self):
        """Тест: только пробелы."""
        result = hamming_distance(" ", " ")
        assert result == 0

    def test_empty_vs_empty_string(self):
        """Тест: пустые строки."""
        result = hamming_distance("", "")
        assert result == 0

    def test_unicode_characters(self):
        """Тест: юникод символы."""
        result = hamming_distance("привет", "привет")
        assert result == 0

    def test_unicode_different(self):
        """Тест: разные юникод символы."""
        result = hamming_distance("привет", "превет")
        assert result == 1

```

```

test_hamming.py::TestHammingDistance::test_both_none PASSED [ 4%]
test_hamming.py::TestHammingDistance::test_first_none PASSED [ 9%]
test_hamming.py::TestHammingDistance::test_second_none PASSED [ 13%]
test_hamming.py::TestHammingDistance::test_different_lengths PASSED [ 18%]
test_hamming.py::TestHammingDistance::test_different_lengths_with_spaces PASSED [ 22%]
test_hamming.py::TestHammingDistance::test_both_empty PASSED [ 27%]
test_hamming.py::TestHammingDistance::test_same_string PASSED [ 31%]
test_hamming.py::TestHammingDistance::test_one_difference PASSED [ 36%]
test_hamming.py::TestHammingDistance::test_two_differences PASSED [ 40%]
test_hamming.py::TestHammingDistance::test_one_difference_hello_hallo PASSED [ 45%]
test_hamming.py::TestHammingDistance::test_all_different PASSED [ 50%]
test_hamming.py::TestHammingDistance::test_case_sensitive PASSED [ 54%]
test_hamming.py::TestHammingDistance::test_numbers_as_strings PASSED [ 59%]
test_hamming.py::TestHammingDistance::test_mixed_characters PASSED [ 63%]
test_hamming.py::TestHammingDistance::test_long_strings PASSED [ 68%]
test_hamming.py::TestHammingDistance::test_special_characters PASSED [ 72%]
test_hamming.py::TestHammingDistanceEdgeCases::test_single_char_same PASSED [ 77%]
test_hamming.py::TestHammingDistanceEdgeCases::test_single_char_different PASSED [ 81%]
test_hamming.py::TestHammingDistanceEdgeCases::test_spaces_only PASSED [ 86%]
test_hamming.py::TestHammingDistanceEdgeCases::test_empty_vs_empty_string PASSED [ 90%]
test_hamming.py::TestHammingDistanceEdgeCases::test_unicode_characters PASSED [ 95%]
test_hamming.py::TestHammingDistanceEdgeCases::test_unicode_different PASSED [100%]

===== 22 passed in 0.04s =====

```

Вывод: освоила приемы тестирования кода на примере использования пакета `pytest`.